Parking Lot Monitoring System

Eduard George Stan

(Github link: https://github.com/EdyStan/parking-lot-monitoring-system)

Overview

The goal of this project is to develop an automatic system for video analysis that enables visual surveillance of on-street parking spaces for a particular scene. The system gets as input data the video stream from a static camera in a specific scene and is able to:

- Classify on-street parking spaces as being occupied or not given a video frame.
- Update configuration of parking spaces given the initial configuration and a video stream to process.
- Track a specific vehicle in the scene of a given video.
- Perform consistently under various environmental conditions, including different weather conditions and varying levels of luminosity.

Table of Contents

1 Key Features

- **Real-time Parking Space Classification:** Automatically classify on-street parking spaces as occupied or free using video analysis.
- **Dynamic Configuration Update:** Ability to update the configuration of parking spaces based on initial setup and real-time video stream data.
- **Vehicle Tracking:** Track specific vehicles across frames in a video feed, enabling detailed surveillance and monitoring.
- **Consistent Results:** Ensure stable and reliable outcomes across varying environmental conditions, including different weather and lighting scenarios.

2 Installation

Requirements

- python = "3.11"
- ultralytics = {git = "https://github.com/THU-MIG/yolov10.git"}
- shapely = "2.0.4"
- opencv-contrib-python = "4.10.0.84"
- huggingface-hub = "0.23.4"

Steps

1. Clone the repository:

```
git clone https://github.com/EdyStan/parking-lot-monitoring-system
```

2. Navigate to the project directory:

```
cd parking-lot-monitoring-system
```

3. Install Poetry. Check out this link: https://python-poetry.org/docs/

4. Install the required dependencies present in `pyproject.toml`:

```
poetry install
```

3 Usage

The file used to run the project is `run_project.py`. There are the following variables that set up the pathing and the preferences of the user:

1. `PROJECT_PATH` - the path where the project is stored in your computer.
2. `TASK{`i`}_PATH` - the directory from which you want to extract the images, for task i.
3. `TASK{`i`}_OUTPUT_PATH` - the directory in which the results are written, for task i.
4. `SHOW_DETAILS` - a boolean variable that specifies whether some additional information will be generated on the run.

4 Implementation

Task 1: The first task consists in correctly classifying the on-street parking spaces listed in the query file as being occupied (label 1) or not (label 0). The on-street parking spaces are numbered as in figure 1a, in the right part of the street, from 1 to 10 from the lower part of the image/scene to the upper part of the image/scene. We do not consider parking places positioned in the left part of the street.

Solution: The first step is to assign the coordinates [1] to the polygons that cover the parking lots of interest, as shown in figure 1. Then, for each image, we read the query file that specifies the parking lots for which we want to extract the label (0 - empty, 1 - occupied). With this information, we create a dictionary, with the keys being extracted from the query file, and the values initialized with 0.

(a) Original indexes

(b) Custom polygons assignment

Figure 1: Unique parking lots identification using polygons.

Next, for each tracking model (`YoloV10n, YoloV8s, YoloV8m`) we compute the bounding boxes of the recognizable objects in the image extracted from the `.jpg` file. After we filter them as being only cars or trucks, we extract their coordinates $\{(x_{min}, y_{min}), (x_{max}, y_{max})\}$ and we check if their average is placed inside one of the parking lot polygons indexed by the keys of the dictionary. If so, we assign value 1 to the considered key. The only thing left to do is to write the output file in the desired format.

Task 2: The task is to list the configuration of the 10 on-street parking places given the initial configuration and a video to process. The initial configuration is represented by a binary vector of length 10, with component i being 0 (free parking space) or 1 (occupied parking space) corresponding to the i-th parking space, as numbered in Figure 1a.

Solution: The solution for Task 2 is, at its core, the same as the one at Task 1. There are only two differences: instead of reading from a `.jpg` file, we use the last frame of a given `.mp4` file, and we need to compute the labels for all parking lots.

Figure 2 displays the same image over which different tracking models were applied. One can notice that these models have better or worse accuracy in terms of detecting vehicles. Even the best model could miss an object that could change the output, and this is why all three models were used.

(a) Bounding given by YoloV10n.

(b) Bounding given by YoloV8s.

(c) Bounding given by YoloV8m.

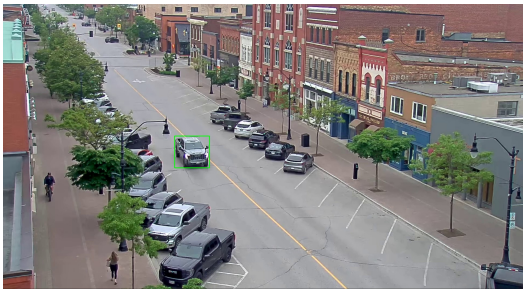
Figure 2: Bounding boxes given by different YOLO models.

Task 3: The task is to track a specific vehicle from the initial frame to the final frame of the video (see figure 3). The initial bounding box of the vehicle to be tracked is provided for the first frame (the annotation follows the format [xmin ymin xmax ymax], where (xmin,ymin) is the top left corner and (xmax,ymax) is the bottom right corner of the initial bounding-box).

Solution: We begin by initializing a list of YOLO models for object detection. For each video, the initial coordinates of the vehicle's bounding box are read from a file. The video frames are then captured and analyzed using the YOLO models to identify potential vehicle bounding boxes. Every $reset\_step = 15$ frames, the CSRT (Channel and Spatial Reliability Tracking) tracker is reinitialized with the bounding box closest to the initial coordinates (that are updated using the YOLO models before each reinitialization of the tracker). For the remaining frames, the tracker updates the vehicle's position, and the coordinates are appended to a trace list.

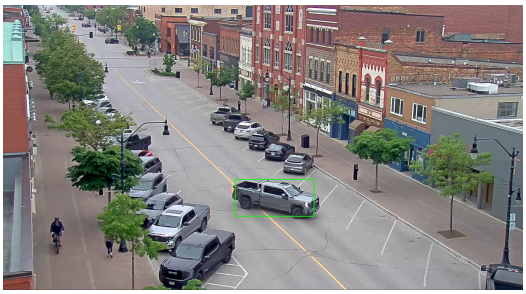
t1

t2



t3

t4



t5

t6

Figure 3: Six frames of a training video containing annotation (green bounding-box) of a specific vehicle.

The reason of this reinitialization at each 15 frames is that the tracker tends to track only a specific part of the vehicle, namely the one that was visible at the initial frame, as shown in figure 4 . To solve this problem, we use the YOLO models to identify the closest vehicle bounding box, with respect to the shifted coordinates given by the tracker.

If `SHOW_DETAILS` is set to True, each frame is displayed with the corresponding green bounding box over the tracked vehicle.

t1

t6

Figure 4: Without the reinitialization using the YOLO models, the tracker mistakenly follows only the part of the vehicle that was visible at the initial frame.

5 Conclusion

In conclusion, the Parking Lot Monitoring System is an exciting, yet challenging project that puts the latest video analysis techniques to practical use. The classical solutions, used initially, with the YOLO models and the CSRT tracker turned out to have a good accuracy on their own ($\backsim 0.8$ for tasks 1 and 2, and $\backsim 0.5$ for task 3). However, by fine-tuning the custom polygons at tasks 1 and 2, and by using an alternation between object detection and tracking at task 3, the accuracy increased significantly ($\backsim 0.99$ for tasks 1 and 2, and $\backsim 0.93$ for task 3).

References

[1] "Image mapping online." https://www.image-map.net/.